

Lecture 08 - Discontinuous Galerkin Methods

Part 2

Georgii Oblapenko

1 Objectives

In this lecture, we will learn about Discontinuous Galerkin Spectral Element methods (DGSEM):

1. Motivation
2. Derivation
3. Entropy-conservative and entropy-stable methods

2 Nodal DG

Modal vs nodal: solution defined as set of coefficients of basis functions vs solution defined as set of values to be interpolated.

2.1 Modal DG recap

Previously, we looked at **nodal** DG, i.e. we assume that our state vector is

$$\hat{u} = \sum_{i=1}^N \sum_{p=0}^K u_p^{(i)} v_p^{(i)} = \sum_{i=1}^N \hat{u}^{(i)},$$

where v_p are basis functions.

Advantages:

- Direct access to modes (for example, in limiting)
- Easy p -adaptivity (just add more modes)

Disadvantages:

- Need to compute values of $\hat{u}$ whenever we need them
- Unsuitable for construction of structure-preserving schemes

2.2 Nodal DG

Given set of K of values u_i at nodes x_i , we can construct a polynomial of degree $K - 1$ that interpolates these values.

The polynomial is given by the Lagrange interpolation polynomial:

$$L_{K-1}(x) = \sum_{i=1}^K u_i l_i(x), \quad l_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

We will be deriving a DG method that operates on the **point values** u_i .

Key property of $l_i(x)$: $l_i(x_j) = \delta_{ij}$.

2.3 Nodal DG: pros and cons

Advantages:

- Direct access to values of u_i
- Construction of entropy-stable schemes possible

Disadvantages:

- p -adaptivity, limiting require explicitly constructing polynomial

2.4 DGSEM

Key idea: take the interpolation nodes x_j to be the quadrature nodes: **collocated** nodes.

Taking l_i as our test functions, we have for $\partial_t u$:

$$\frac{d}{dt} \int_{\Omega} u v dx \approx \frac{d}{dt} \sum_{j=1}^K \omega_j u(x_j) l_i(x_j) = \frac{d}{dt} \omega_i u_i.$$

So our mass matrix is diagonal.

Question

Where does the error come from here? Why “approximate” sign?

Collocation-induced error

In case we use GLL nodes, we have polynomials of degree $2K - 2$, but with K GLL nodes we integrate exactly only up to $2K - 3$!

2.5 Strong form DG

With a numerical flux $g(u_{ij}, \mathbf{n}_{ij}) = g(u(\mathbf{x}_{ij}), u(\mathbf{x}_{ji}), \mathbf{n}_{ij})$ we obtain the following weak form of the DG scheme

$$\partial_t \int_{\Omega_i} u v dx + \sum_{j \in \mathcal{N}(i)} \int_{\partial\Omega_{ij}} g(u_{ij}, \mathbf{n}_{ij}) v dS - \int_{\Omega_i} \mathbf{f}(u) \cdot \nabla v dV = 0$$

Let's integrate by parts again!

Strong form

$$\partial_t \int_{\Omega_i} u v dx + \sum_{j \in \mathcal{N}(i)} \int_{\partial\Omega_{ij}} (g(u_{ij}, \mathbf{n}_{ij}) - \mathbf{f}(u(\mathbf{x}_{ij})) \cdot \mathbf{n}_{ij}) v dS + \int_{\Omega_i} (\nabla \cdot \mathbf{f}(u)) v dV = 0$$

2.6 Why strong form DG?

$$\partial_t \int_{\Omega_i} u v dx + \sum_{j \in \mathcal{N}(i)} \int_{\partial\Omega_{ij}} (g(u_{ij}, \mathbf{n}_{ij}) - \mathbf{f}(u(\mathbf{x}_{ij})) \cdot \mathbf{n}_{ij}) v dS + \int_{\Omega_i} (\nabla \cdot \mathbf{f}(u)) v dV = 0$$

We have clean separation of volume and surface terms. For smooth solution (no jumps at interfaces), surface term disappears.

Question

Are strong/weak forms always equivalent?

2.7 The SBP property

In continuous setting, integration by parts always holds. In discrete setting, **not always**.

Discrete version is called **Summation By Parts (SBP)**. It is a property of numerical integration/differentiation methods, not a theorem.

From this point, some things will be done in 1D, multi-dimensional schemes are usually tensor-products of 1D schemes.

2.8 Derivative matrix

We can write the derivative operator as a matrix:

$$\partial_x u = \partial_x \sum_{i=1}^K u_i l_i(x) = \sum_{i=1}^K u_i \partial_x l_i(x).$$

Since our integration points are the same as the nodes of the Lagrange basis, we don't need to interpolate the derivative.

$$\partial_x u(x_j) = \sum_{i=1}^K u_i \partial_x l_i(x_j) \Leftrightarrow \partial_x u = Du, \quad D_{ji} = \partial_x l_i(x_j).$$

2.9 Mass matrix

We also write out the mass matrix M :

$$M_{ij} = \sum w_k \delta_{ik} \delta_{jk},$$

this defines the product of $\underline{\mathbf{u}} = (u_1, \dots, u_K)$ and $\underline{\mathbf{v}} = (v_1, \dots, v_K)$ as

$$\underline{\mathbf{u}}^T M \underline{\mathbf{v}}.$$

So for the time derivative of the nodal values $\underline{\mathbf{u}} = (u_1, \dots, u_K)$ we obtain

$$M \frac{d\underline{\mathbf{u}}}{dt} = RHS.$$

2.10 SBP property

If

$$MD + (MD)^T = B, \quad B = \text{diag}(-1, 0, \dots, 0, 1),$$

this is called the summation-by-parts property.

Translating to nodal values of functions:

$$\underline{\mathbf{u}}^T M(D\underline{\mathbf{v}}) = \underline{\mathbf{u}}^T B \underline{\mathbf{v}} - (D\underline{\mathbf{u}})^T M \underline{\mathbf{v}}.$$

Gauss-Lobatto nodes have the SBP property!

Generalization to arbitrary elements:

- boundary mass matrix B
- normal multiplication operators N_j (for coordinate j)
- restriction operator R

2.11 SBP semidiscretization

Omitting any Jacobians (due curved elements), we obtain for the weak form ($\underline{\cdot}$ denotes vector of nodal values):

$$\partial_t \underline{\mathbf{u}} = -M^{-1} B \underline{\mathbf{g}} + M^{-1} D^T M \underline{\mathbf{f}}.$$

Using SBP (and M is diagonal), re-group and obtain strong form

$$\partial_t \underline{\mathbf{u}} = -M^{-1} B (\underline{\mathbf{g}} - \underline{\mathbf{f}}) - D \underline{\mathbf{f}}.$$

2.12 Stability

Want provable stability properties for our numerical scheme!

- [Jian, Shu](#) - non-linear stability for **scalar** conservation equations assuming **exact** evaluation of integrals
- What about **systems** of equations?

💡 Question

Why does **exact** integration assumption not work out in practice for Euler equations?

i Exact integration failure

For Euler equations, fluxes involve **rational** functions of the conserved variables (for example, $\rho v_x v_y$), which means that **exact** integration is not possible.

3 Entropy conservative schemes

Even stable solutions can be **non-physical**!

What does **physical** mean exactly?

3.1 Entropy

Convex function $s(\mathbf{u}) : \mathbb{R}^m \rightarrow \mathbb{R}$ is called entropy, if

$$\exists F_j(\mathbf{u}) \text{ s.t. } \partial_{\mathbf{u}} s(\mathbf{u}) \partial_{\mathbf{u}} \mathbf{f}_j = \partial_{\mathbf{u}} F_j(\mathbf{u}), \quad j = 1, \dots, d.$$

$\partial_{\mathbf{u}} s(\mathbf{u})$, $\partial_{\mathbf{u}} F_j(\mathbf{u})$ - vectors of length m , $\partial_{\mathbf{u}} \mathbf{f}_j$ - flux Jacobian.

Example for Euler equations: $s = \log p - \gamma \log \rho$. Often distinguish *physical* entropy s_{phys} and *mathematical* entropy $s_{math} = -\frac{\rho s}{\gamma-1}$.

3.2 Second law (physical entropy)

From second law of thermodynamics, we can define entropy for reversible processes:

$$ds = \frac{\delta Q}{T},$$

and we have for general processes

$$\Delta s \geq 0.$$

For an ideal gas, expression for entropy can be derived via statistical mechanics.

3.3 Entropy variables

We want numerical scheme to respect laws of thermodynamics (or at least violate them as little as possible and in predictable ways). Introduce $\mathbf{w} = \partial_{\mathbf{u}} s(\mathbf{u})$ - entropy variables.

We have

1. $\partial_t s(\mathbf{u}) = \mathbf{w}^T \partial_t \mathbf{u}$
2. $\mathbf{w}^T \partial_{\mathbf{u}} \mathbf{f}_j = \partial_{\mathbf{u}} F_j(\mathbf{u})$

3.4 Entropy inequality

Multiplying $\partial_t \mathbf{u} + \nabla \cdot \mathbf{f} = 0$ by $\mathbf{w}^T$ we get

$$\partial_t s + \nabla \cdot \mathbf{F} = 0.$$

This holds for smooth solutions.

For discontinuous solutions (irreversible processes), we have (for *mathematical* entropy)

$$\int_{\Omega} \partial_t s + \int_{\partial\Omega} \mathbf{F} \cdot \mathbf{n} dS \leq 0.$$

! Important

This hinges on 1) chain rule 2) integration by parts! For discrete version, this is why we **need SBP** and **special fluxes**.

3.5 Entropy-stability analysis in “classical DG”

For Euler equations in 1D, entropy variables are

$$\mathbf{w} = \left(\frac{1}{\gamma - 1} (\ln p - \gamma \ln \rho) - \frac{\rho v^2}{2p}, \frac{\rho v}{p}, -\frac{\rho}{p} \right)^T.$$

Non-polynomial functions of state variables, cannot perform an analytical analysis.

Possible approach (Hughes et al., Hildebrand et al.): solve equations directly in $\mathbf{w}$. Involves implicit dependencies of $\mathbf{u}$ on $\mathbf{w}$.

i Note

How do we ensure non-violation of entropy inequalities in a “cleaner”/“easier” fashion? We need equivalent of discrete chain rule.

3.6 Flux-differencing

Semi-discretization:

$$\partial_t \underline{\mathbf{u}} = -M^{-1} B \underline{\mathbf{g}} + M^{-1} D^T M \underline{\mathbf{f}}.$$

Start by noticing that every row of D sums up to 0, i.e. $\sum_{k=1}^K D_{j,k} = 0$. Re-write

$$(D \underline{\mathbf{f}})_j = \sum_{k=1}^K D_{j,k} \mathbf{f}_k + \mathbf{f}_j \sum_{k=1}^K D_{j,k} = 2 \sum_{k=1}^K \frac{1}{2} D_{j,k} (\mathbf{f}_j + \mathbf{f}_k).$$

💡 Question

What is $\frac{1}{2}(\mathbf{f}_j + \mathbf{f}_k)$?

i Note

Nothing else than the central flux!

3.7 Flux-differencing

Key idea: **replace** $\mathbf{f}_j^{central} = \frac{1}{2}(\mathbf{f}_j + \mathbf{f}_k)$ with $\mathbf{f}^{EC}$.

Define **potential flux** (j is coordinate):

$$\psi_j(\mathbf{w}) = \mathbf{w}^T \mathbf{f}_j(\mathbf{u}(\mathbf{w})) - \mathbf{F}_j, j = 1, \dots, d$$

Tadmor's entropy conservation condition on flux function $\mathbf{f}_{j,S}$:

$$(\mathbf{w}_R - \mathbf{w}_L)^T \mathbf{f}_{j,EC}(\mathbf{u}_L, \mathbf{u}_R) = \psi_j(\mathbf{w}_R) - \psi_j(\mathbf{w}_L).$$

If $(\mathbf{w}_R - \mathbf{w}_L)^T \mathbf{f}_{j,EC}(\mathbf{u}_L, \mathbf{u}_R) \leq \psi_j(\mathbf{w}_R) - \psi_j(\mathbf{w}_L)$, flux is **entropy-stable**.

3.8 Entropy-conservation/stability of DG method

If

- $\mathbf{f}_{EC}^{vol}$ is consistent with f , symmetric, EC
- M is diagonal, B is diagonal
- $\mathbf{f}^{surf}$ is EC/ES

then semi-discretization

$$\partial_t \underline{\mathbf{u}} = -M^{-1} B (\underline{\mathbf{f}} - \underline{\mathbf{f}}^{surf}) - 2D \underline{\mathbf{f}}_{EC}^{vol}$$

is EC/ES (depending on numerical surface flux).

3.9 EC fluxes: derivation

1. Choose primitive variables $\mathbf{z}$
2. Compute flux potentials, entropy, entropy variables in terms of $\mathbf{z}$
3. Plug into $[[\mathbf{w}]] \cdot \mathbf{f}_{EC} - [[\psi]] = 0$
4. Group terms in front of each $[[z_i]]$ and set to 0
5. Solve for components of $\mathbf{f}_{EC}$

EC fluxes are not unique!

3.10 EC fluxes: computational cost

On a tensor-product grid:

- “Classical” volume flux term: $d(K+1)^d$ operations per cell
- Flux-differencing flux term: $dK/2(K+1)^d$ operations per cell

EC fluxes also in general more computationally expensive, as they usually involve logarithms (for Euler equations); approximately 50 % slower than simple flux evaluation.

3.11 ES fluxes: LLF/Rusanov

For $\mathbf{f}^{surf}$, use a **modified** Rusanov flux: instead of

$$\mathbf{f}^{surf}(\mathbf{u}_L, \mathbf{u}_R) = \frac{1}{2} (\mathbf{f}(\mathbf{u}_L) + \mathbf{f}(\mathbf{u}_R) - \lambda(\mathbf{u}_R - \mathbf{u}_L)), \quad \lambda = \max EV(|\partial \mathbf{f} / \partial \mathbf{u}|)$$

we take

$$\mathbf{f}^{surf}(\mathbf{u}_L, \mathbf{u}_R) = \mathbf{f}^{EC}(\mathbf{u}_L, \mathbf{u}_R) - \frac{1}{2} (\lambda(\mathbf{u}_R - \mathbf{u}_L)).$$

i Note

For Euler equations, if density flux has **no contribution of pressure**, the LLF scheme + forward Euler ensure non-negative density under a suitable CFL condition $\implies$ SSPRK will preserve non-negative densities.

4 Sub-cell FV/DG blending as limiter

(Very briefly)

- Treat GLL nodes as defining FV grid
- Based on some shock indicator (proportion of energy in highest modes) compute smoothly varying α
- Blend DG and low-order FV approach: $\dot{\mathbf{u}} = \alpha \dot{\mathbf{u}}^{FV} + (1 - \alpha) \dot{\mathbf{u}}^{DG}$
- Extensions: use 1-st order FV; find optimal α to ensure positivity of pressure, density

5 Summary

- Nodal DG allows for not only for alternative viewpoint, but also to construct SBP schemes
- SBP property allows to convert a scheme into a flux-differencing scheme
- Flux-differencing schemes allow to construct EC/ES schemes

6 Questions

- What is the difference between nodal and modal DG? What is the key point of DGSEM, and which error it induces when used with Gauss-Lobatto-Legendre nodes?
- What is the summation-by-parts property, and how can it be used to construct a flux-differencing scheme?
- What is entropy, why is it important, and how does one use SBP-DGSEM to construct an entropy conservative/stable scheme?

7 References

- T. Chen, C.-W. Shu, Review of Entropy Stable Discontinuous Galerkin Methods for Systems of Conservation Laws on Unstructured Simplex Meshes
- G. J. Gassner, A. R. Winters, A Novel Robust Strategy for Discontinuous Galerkin Methods in Computational Fluid Mechanics: Why? When? What? Where?
- H. Ranocha, Comparison of Some Entropy Conservative Numerical Fluxes for the Euler Equations